



PROJECT REPORT

PROJECT II – IT3930

Facility location

Trần Tuấn Kiệt

Student ID: 20235132

Kiet.TT235132@sis.hust.edu.vn

Supervisors

Dr. Nguyễn Khánh Phương

Hanoi, 2026

Mục lục

1	Thiết kế cấu trúc dữ liệu cơ bản bài toán vị trí đặt kho	4
1.1	Giới thiệu bài toán	4
1.2	Đối tượng Kho (Facility)	5
1.2.1	Các thuộc tính tĩnh	5
1.2.2	Các thuộc tính động	5
1.3	Đối tượng Khách hàng (Customer)	5
1.3.1	Các thuộc tính tĩnh	5
1.3.2	Các thuộc tính động	5
2	Mô hình toán học (MILP)	6
2.1	Tập chỉ số và Tham số	6
2.2	Biến quyết định	6
2.3	Hàm mục tiêu	6
2.4	Hệ thống các ràng buộc	7
2.4.1	Ràng buộc phục vụ khách hàng	7
2.4.2	Ràng buộc sức chứa của cơ sở	7
2.4.3	Ràng buộc chỉ gán cho cơ sở đã mở	7
2.4.4	Miền giá trị của biến	7
3	Phương pháp tiếp cận tham lam (Greedy algorithms)	8
3.1	Chiến lược Greedy 1: Ưu tiên mở cơ sở "đáng tiền nhất"	8
3.1.1	Ý tưởng thuật toán và tiêu chí tỷ lệ lợi ích/chi phí	8
3.1.2	Ưu điểm và nhược điểm của Greedy 1	9
3.2	Chiến lược Greedy 2: Ưu tiên gán khách hàng khó trước	9
3.2.1	Ý tưởng thuật toán và các tiêu chí đánh giá "mức khó"	10
3.2.2	Ưu điểm và nhược điểm của Greedy 2	10
4	Các phương pháp cải thiện nghiệm	12
4.1	Các bước di chuyển (Moves) cải thiện nghiệm	12
4.1.1	Move 1: Reassignment (Gán lại khách hàng)	12
4.1.2	Move 2: Close facility (Đóng cơ sở)	13
4.1.3	Move 3: Open + reassign (Mở và chuyển khách hàng)	13
4.1.4	Move 4: Swap (Hoán đổi trạng thái đóng/mở)	13
4.2	Sơ đồ thuật toán Local Search	13
4.3	Tối ưu hóa tính toán: Phương pháp tính nhanh Delta	14
5	Cài đặt và thực nghiệm	15
5.1	Mô tả bộ dữ liệu chuẩn	15

5.2	Cài đặt mô hình Exact MILP sử dụng thư viện OR-Tools	16
5.3	Cài đặt các thuật toán Greedy	16
5.4	Cài đặt thuật toán Local Search	16
5.5	Quy trình chạy thực nghiệm và Tổng hợp kết quả	17
6	Đánh giá và so sánh kết quả	18
6.1	So sánh chất lượng nghiệm giữa Exact MILP, Greedy và Local Search	18
6.1.1	So sánh MILP và Greedy	18
6.1.2	So sánh Greedy 2 và các phương pháp Local Search	19
6.1.3	Thống kê số instance đạt nghiệm tốt nhất và mức cải thiện	19
6.2	So sánh thời gian thực thi của các phương pháp	19
6.3	Thảo luận về hiệu quả các bước Local Search và phương pháp tính nhanh .	20
6.4	Kết luận chương	20
7	Kết luận và hướng phát triển	22
7.1	Tóm tắt toàn bộ đồ án	22
7.2	Đóng góp chính của đồ án	23
7.3	Hạn chế của đồ án	23
7.4	Hướng phát triển trong tương lai	23
7.4.1	Nâng cấp khung Local Search hiện tại	24
7.4.2	Áp dụng các Metaheuristic nâng cao	24
7.4.3	Tiếp cận lại (Matheuristics) và mở rộng phân tích	24

Chương 1

Thiết kế cấu trúc dữ liệu cơ bản bài toán vị trí đặt kho

1.1 Giới thiệu bài toán

Bài toán vị trí đặt kho có ràng buộc sức chứa (Capacitated Facility Location Problem - CFLP) bắt nguồn từ nhu cầu thực tế trong việc thiết kế mạng lưới chuỗi cung ứng. Ngữ cảnh điển hình là một công ty phân phối hàng tiêu dùng đang muốn xây dựng mạng lưới kho trung chuyển để cung cấp hàng cho các cửa hàng bán lẻ trong một thành phố.

Công ty đã khảo sát trước một tập hợp các vị trí ứng viên có thể mở kho. Mỗi vị trí ứng viên đặc trưng bởi một chi phí mở kho cố định và một sức chứa tối đa về lượng hàng có thể phục vụ trong một kỳ. Phân bố khắp thành phố là các cửa hàng bán lẻ (đóng vai trò là khách hàng), trong đó mỗi cửa hàng có một mức nhu cầu hàng hóa nhất định. Đồng thời, tùy thuộc vào khoảng cách và điều kiện giao thông, sẽ phát sinh một mức chi phí phục vụ tương ứng nếu cửa hàng đó được cung cấp hàng từ một kho ứng viên cụ thể.

Hệ thống cần đưa ra quyết định cốt lõi: nên mở những kho nào và mỗi cửa hàng sẽ được phục vụ bởi kho nào. Quá trình phân bổ này phải tuân thủ nghiêm ngặt hai ràng buộc:

- Mỗi cửa hàng được phục vụ bởi đúng một kho đã mở.
- Tổng nhu cầu của các cửa hàng được giao cho một kho không được phép vượt quá sức chứa tối đa của kho đó.

Mục tiêu tối thượng của mô hình là đảm bảo tổng chi phí mở kho và chi phí phục vụ đạt mức nhỏ nhất.

Để có thể cài đặt và thực thi hiệu quả các thuật toán tối ưu hóa (như thuật toán Tham lam hay Tìm kiếm cục bộ) cho bài toán này, việc tổ chức cấu trúc dữ liệu linh hoạt là yêu cầu tiên quyết. Dưới đây là thiết kế chi tiết cho hai đối tượng cốt lõi: Kho (Facility) và Khách hàng (Customer).

1.2 Đối tượng Kho (Facility)

Đối tượng này đại diện cho một vị trí ứng viên có thể mở kho trung chuyển. Việc quản lý tốt trạng thái của Kho sẽ giúp thuật toán kiểm tra nhanh chóng các ràng buộc về sức chứa.

1.2.1 Các thuộc tính tĩnh

Đây là các giá trị được khởi tạo một lần khi đọc từ file dữ liệu và không thay đổi trong suốt quá trình chạy thuật toán:

- **ID (Chỉ số):** Định danh duy nhất của kho ($i \in \{0, 1, \dots, I - 1\}$).
- **Capacity (Sức chứa tối đa - s_i):** Lượng hàng hóa tối đa mà kho có thể phục vụ trong một kỳ.
- **Fixed Cost (Chi phí mở kho - f_i):** Chi phí cố định phải bỏ ra nếu quyết định mở kho này.

1.2.2 Các thuộc tính động

- **Is Open (Trạng thái mở):** Biến logic xác định kho hiện tại đang được mở hay đóng tương đương với biến quyết định $y_i \in \{0, 1\}$.
- **Current Load (Tải trọng hiện tại):** Tổng nhu cầu của tất cả các khách hàng đang được gán cho kho này. Thuộc tính này dùng để kiểm tra điều kiện (Current Load + Nhu cầu khách mới $\leq$ Capacity, $\sum d_j * x_{ij} \leq s_i$) trước khi gán thêm khách hàng.
- **Residual Capacity (Sức chứa còn lại):** Bằng Capacity trừ đi Current Load. (Có thể tính toán trực tiếp hoặc lưu trữ riêng để truy xuất nhanh, $s_i - \sum d_j * x_{ij}$).
- **Assigned Customers (Danh sách khách hàng đang phục vụ):** Tập hợp các ID của khách hàng đang được gán cho kho này.

1.3 Đối tượng Khách hàng (Customer)

Đối tượng này đại diện cho một cửa hàng bán lẻ cần được cung cấp hàng hóa.

1.3.1 Các thuộc tính tĩnh

- **ID (Chỉ số):** Định danh duy nhất của khách hàng (ví dụ: $j \in \{0, 1, \dots, J - 1\}$).
- **Demand (Nhu cầu - d_j):** Lượng hàng hóa mà khách hàng này yêu cầu.
- **Serving Costs (Chi phí phục vụ - c_{ij}):** Một danh sách hoặc bản đồ (Map) lưu trữ chi phí vận chuyển từ TẤT CẢ các kho i đến khách hàng j này.

1.3.2 Các thuộc tính động

- **Assigned Facility (Kho đang phục vụ):** ID của kho mà khách hàng này hiện đang được gán tới (tương đương với việc tìm ra biến $x_{ij} = 1$).

Chương 2

Mô hình toán học (MILP)

Để giải quyết bài toán một cách chính xác, chúng ta có thể mô hình hóa bài toán Vị trí đặt kho có giới hạn sức chứa (Capacitated Facility Location Problem) dưới dạng Quy hoạch tuyến tính nguyên hỗn hợp (Mixed-Integer Linear Programming - MILP).

2.1 Tập chỉ số và Tham số

Mô hình sử dụng các tập chỉ số và tham số đầu vào như sau:

- I : tập cơ sở (kho) ứng viên, với chỉ số $i \in I$.
- J : tập khách hàng (cửa hàng bán lẻ), với chỉ số $j \in J$.
- f_i : chi phí cố định để mở cơ sở i .
- s_i : sức chứa tối đa của cơ sở i trong một kỳ].
- d_j : lượng hàng hóa nhu cầu của khách hàng j .
- c_{ij} : chi phí phục vụ (vận chuyển) để cung cấp hàng cho khách hàng j từ cơ sở i .

2.2 Biến quyết định

Bài toán yêu cầu xác định việc mở kho nào và phân bổ khách hàng ra sao, do đó sử dụng hai cụm biến quyết định nhị phân:

- $y_i \in \{0, 1\}$: bằng 1 nếu quyết định mở cơ sở i , ngược lại bằng 0.
- $x_{ij} \in \{0, 1\}$: bằng 1 nếu khách hàng j được gán (phục vụ) bởi cơ sở i , ngược lại bằng 0.

2.3 Hàm mục tiêu

Mục tiêu cốt lõi của bài toán là tối thiểu hóa tổng chi phí, bao gồm tổng chi phí mở cơ sở và tổng chi phí phục vụ khách hàng.

$$\min \sum_{i \in I} f_i y_i + \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} \quad (2.1)$$

2.4 Hệ thống các ràng buộc

Để đảm bảo tính hợp lệ của hệ thống thực tế, mô hình toán học phải thỏa mãn các ràng buộc sau đây:

2.4.1 Ràng buộc phục vụ khách hàng

Mỗi khách hàng phải được gán cho đúng một cơ sở duy nhất.

$$\sum_{i \in I} x_{ij} = 1 \quad \forall j \in J \quad (2.2)$$

2.4.2 Ràng buộc sức chứa của cơ sở

Tổng nhu cầu của tất cả các khách hàng được gán cho một cơ sở không được vượt quá sức chứa tối đa của cơ sở đó. Đồng thời, ràng buộc này cũng đảm bảo nếu cơ sở không mở ($y_i = 0$), sức chứa sẽ bằng 0 và không khách hàng nào được gán vào.

$$\sum_{j \in J} d_j x_{ij} \leq s_i y_i \quad \forall i \in I \quad (2.3)$$

2.4.3 Ràng buộc chỉ gán cho cơ sở đã mở

Một khách hàng chỉ có thể được gán cho một cơ sở nếu cơ sở đó đã được mở. Mặc dù ràng buộc này không bắt buộc tuyệt đối nếu đã có ràng buộc sức chứa ở trên (với $d_j > 0$), nhưng việc giữ lại ràng buộc này giúp mô hình chặt chẽ rõ ràng hơn và các bộ giải (solver) thường xử lý tốt hơn.

$$x_{ij} \leq y_i \quad \forall i \in I, \forall j \in J \quad (2.4)$$

2.4.4 Miền giá trị của biến

Các biến quyết định đều là biến nguyên nhị phân.

$$x_{ij} \in \{0, 1\}, \quad y_i \in \{0, 1\} \quad \forall i \in I, \forall j \in J \quad (2.5)$$

Chương 3

Phương pháp tiếp cận tham lam (Greedy algorithms)

Trong bài toán định vị cơ sở có ràng buộc sức chứa (Capacitated Facility Location Problem - CFLP), mục tiêu là lựa chọn một tập cơ sở để mở và phân bổ khách hàng cho các cơ sở đó sao cho tổng chi phí là nhỏ nhất. Tổng chi phí này bao gồm chi phí cố định khi mở cơ sở và chi phí phục vụ khách hàng từ cơ sở được chọn.

3.1 Chiến lược Greedy 1: Ưu tiên mở cơ sở "đáng tiền nhất"

Chiến lược Greedy 1 được xây dựng theo tư tưởng tham lam, tại mỗi bước thuật toán sẽ chọn cơ sở mang lại lợi ích tốt nhất ở thời điểm hiện tại. Cụ thể, thuật toán ưu tiên mở cơ sở có tỷ lệ lợi ích trên chi phí cao nhất, tức là cơ sở "đáng tiền nhất" để mở trong bước đó. Thay vì xét toàn bộ không gian nghiệm như mô hình tối ưu chính xác, phương pháp này ra quyết định từng bước dựa trên thông tin cục bộ. Sau khi mở một cơ sở, nhóm khách hàng chưa được phục vụ sẽ được gán cho cơ sở đó nếu còn đủ sức chứa, và quá trình lặp lại cho đến khi toàn bộ khách hàng được phân bổ.

3.1.1 Ý tưởng thuật toán và tiêu chí tỷ lệ lợi ích/chi phí

Ý tưởng chính của Greedy 1 là đánh giá từng cơ sở chưa mở dựa trên khả năng phục vụ các khách hàng còn lại.

Quy trình tổng quát được thực hiện như sau:

- Khởi tạo tất cả khách hàng ở trạng thái chưa được phân bổ và tất cả cơ sở ở trạng thái chưa mở.
- Với mỗi cơ sở chưa mở, xét các khách hàng chưa được phục vụ và lọc ra các khách hàng có nhu cầu không vượt quá sức chứa của cơ sở.
- Sắp xếp các khách hàng theo chi phí phục vụ tăng dần và chọn lần lượt cho đến khi hết sức chứa.
- Tính tỷ lệ lợi ích/chi phí của cơ sở đó và chọn cơ sở có tỷ lệ tốt nhất để mở.
- Gán các khách hàng đã chọn cho cơ sở vừa mở, lặp lại quá trình cho đến khi tất cả khách hàng được phân bổ.

Thuật toán ước lượng lợi ích của một cơ sở thông qua phần chi phí phục vụ tiết kiệm được. Mức tiết kiệm này là chênh lệch giữa chi phí phục vụ từ cơ sở đang xét với chi phí phục vụ lớn nhất của khách hàng đó trong toàn bộ các cơ sở. Công thức tính lợi ích và chi phí:

- Lợi ích: $saved_cost(i) = \sum (\max_k(c_{kj}) - c_{ij})$ với mọi khách hàng j thuộc tập được chọn tạm thời S_i .
- Chi phí: $cost(i) = f_i + \sum c_{ij}$ với f_i là chi phí cố định và c_{ij} là chi phí phục vụ.
- Tiêu chí lựa chọn (Tỷ lệ): $ratio(i) = \frac{saved_cost(i)}{cost(i)}$.

Mỗi quyết định mở cơ sở và gán khách hàng được thực hiện một lần và không quay lại chỉnh sửa, do đó nghiệm thu được phụ thuộc mạnh vào các lựa chọn ban đầu.

3.1.2 Ưu điểm và nhược điểm của Greedy 1

Ưu điểm:

- Thuật toán có cách triển khai đơn giản, dễ hiểu, dễ cài đặt để làm nền tảng cho các phương pháp cải tiến.
- Thời gian chạy nhanh hơn rất nhiều so với mô hình tối ưu chính xác (MILP), hữu ích khi kích thước bài toán lớn.
- Tiêu chí đánh giá giúp cân bằng giữa chi phí mở cơ sở và chi phí phục vụ khách hàng, tránh việc mở quá nhiều cơ sở đắt đỏ.
- Khả năng tạo ra nghiệm ban đầu khả thi để dùng làm đầu vào cho các thuật toán tìm kiếm cục bộ (Move 1, 2, 3, 4 hoặc VND).

Nhược điểm:

- Tính cục bộ trong quyết định rất cao, không xét đến ảnh hưởng của lựa chọn hiện tại tới các bước sau.
- Không có cơ chế sửa sai khi đã lỡ gán khách hàng, để làm nghiệm cuối cùng bị mắc kẹt.
- Tiêu chí tỷ lệ tính toán chỉ là thước đo xấp xỉ, không đảm bảo sẽ dẫn đến nghiệm toàn cục tốt nhất.
- Việc sắp xếp theo chi phí phục vụ tăng dần có thể bỏ qua yếu tố nhu cầu và sức chứa, gây khó khăn cho việc phân bổ dài hạn.
- Chất lượng nghiệm thường kém hơn so với Greedy 2 và các phương pháp cải thiện khác.

3.2 Chiến lược Greedy 2: Ưu tiên gán khách hàng khó trước

Một khó khăn trong bài toán CFLP là nếu các khách hàng có nhu cầu lớn không được xử lý sớm, chúng có thể trở nên rất khó gán ở các bước sau vì sức chứa của cơ sở đã bị chia nhỏ. Greedy 2 giải quyết vấn đề này bằng tư tưởng ưu tiên xử lý các khách hàng "khó" (có nhu cầu lớn) trước.

Khác với việc chọn cơ sở để mở trước như Greedy 1, Greedy 2 duyệt từng khách hàng theo thứ tự giảm dần của nhu cầu và chọn ra cơ sở có chi phí gán nhỏ nhất. Chi phí gán cân nhắc đồng thời cả chi phí phục vụ và chi phí cố định để mở cơ sở (nếu cơ sở đó chưa mở).

3.2.1 Ý tưởng thuật toán và các tiêu chí đánh giá "mức khó"

Tiêu chí "độ khó" trong phiên bản này được đánh giá trực tiếp qua nhu cầu d_j của khách hàng, tức là khách hàng có d_j càng lớn sẽ càng được xử lý trước. Xử lý khách hàng lớn trước giúp giảm thiểu nguy cơ tạo ra nghiệm không khả thi khi sức chứa là tài nguyên bị giới hạn.

Quy trình hoạt động của Greedy 2:

- Khởi tạo khách hàng, cơ sở và sức chứa ban đầu, sau đó sắp xếp khách hàng theo nhu cầu giảm dần.
- Xét tuần tự từng khách hàng và kiểm tra tất cả các cơ sở còn đủ sức chứa.
- Tính chi phí gán khách hàng vào từng cơ sở hợp lệ, bao gồm cả chi phí mở kho nếu kho chưa hoạt động.
- Lựa chọn cơ sở có chi phí gán nhỏ nhất, mở cơ sở (nếu cần), cập nhật chi phí tổng và sức chứa còn lại của cơ sở.
- Thuật toán dừng lại và kết luận không khả thi nếu tại một bước không còn cơ sở nào đủ sức chứa.

Chi phí gán một khách hàng j vào cơ sở i được tính bằng c_{ij} nếu cơ sở đã mở, và bằng $c_{ij} + f_i$ nếu cơ sở chưa mở. Cơ sở tốt nhất cho khách hàng j là cơ sở thỏa mãn điều kiện sức chứa và có giá trị $assignment_cost(i, j)$ đạt cực tiểu. Việc kết hợp 2 tiêu chí (ưu tiên khách hàng lớn và chọn cơ sở chi phí rẻ nhất) giúp đảm bảo tính khả thi đồng thời giữ tổng chi phí ở mức thấp.

3.2.2 Ưu điểm và nhược điểm của Greedy 2

Ưu điểm:

- Khả năng xử lý các ràng buộc về sức chứa tốt hơn hẳn bằng việc ưu tiên các khách hàng có nhu cầu lớn.
- Có thời gian chạy nhanh và đơn giản, cực kì phù hợp tạo nghiệm ban đầu cho thuật toán cải thiện.
- Việc cộng thêm chi phí cố định vào chi phí gán giúp hạn chế mở thêm cơ sở mới một cách tùy tiện.
- Cung cấp cấu trúc nghiệm rõ ràng, rành mạch để dễ dàng triển khai các phép tìm kiếm cục bộ (Move 1, 2, 3, 4 hoặc VND).
- Trong thực nghiệm, kết quả trả về của Greedy 2 thường ưu việt hơn so với Greedy 1 nhờ cách giải quyết trực tiếp ràng buộc sức chứa.

Nhược điểm:

- Thuật toán vẫn dựa trên các quyết định tham lam cục bộ tại từng bước.

- Mức độ "khó" hiện tại chỉ dựa vào nhu cầu, bỏ qua các yếu tố quan trọng khác như số lượng cơ sở có thể phục vụ hay chênh lệch chi phí.
- Hoàn toàn không có cơ chế tự điều chỉnh và chuyển đổi khách hàng sau khi đã gán.
- Đánh giá toàn bộ chi phí cố định cho khách hàng đầu tiên đôi khi gây ra sự thiên vị cho các cơ sở đã được mở sẵn.
- Chất lượng của nghiệm cuối cùng chịu sự tác động lớn từ thứ tự sắp xếp của các khách hàng có nhu cầu tương đương nhau.

Chương 4

Các phương pháp cải thiện nghiệm

Sau khi xây dựng được nghiệm ban đầu bằng thuật toán Greedy 2, nghiệm thu được thường chưa phải là nghiệm tốt nhất. Nguyên nhân là do Greedy 2 đưa ra quyết định tuần tự theo từng khách hàng và không quay lại điều chỉnh các lựa chọn trong quá khứ. Vì vậy, việc áp dụng các bước di chuyển cục bộ (Local Search) là cần thiết để cải thiện chất lượng nghiệm.

4.1 Các bước di chuyển (Moves) cải thiện nghiệm

Các bước di chuyển (Moves) là các phép biến đổi nhỏ trên nghiệm hiện tại nhằm tạo ra một nghiệm lân cận bằng cách thay đổi phân bổ khách hàng hoặc trạng thái đóng/mở của cơ sở. Một phép di chuyển chỉ được thuật toán chấp nhận nếu thỏa mãn đồng thời hai điều kiện: không vi phạm ràng buộc về sức chứa và mức thay đổi chi phí $\Delta < 0$ (nghĩa là tổng chi phí mới nhỏ hơn tổng chi phí hiện tại). Dự án tập trung triển khai 4 loại move cơ bản sau đây.

4.1.1 Move 1: Reassignment (Gán lại khách hàng)

Move 1 thực hiện cải thiện nghiệm bằng cách chuyển một khách hàng j từ cơ sở đang phục vụ (old_fac) sang một cơ sở khác (new_fac). Phép chuyển này chỉ hợp lệ nếu sức chứa còn lại của cơ sở mới lớn hơn hoặc bằng nhu cầu của khách hàng j . Giá trị Δ của Move 1 được tính bằng độ chênh lệch chi phí phục vụ ($c_{new_fac,j} - c_{old_fac,j}$). Ngoài ra:

- Nếu cơ sở mới chưa mở, Δ được cộng thêm chi phí cố định để mở nó ($+f_{new_fac}$).
- Nếu sau khi rút khách hàng j đi, cơ sở cũ không còn phục vụ ai, nó sẽ được đóng cửa và Δ được giảm trừ đi phần chi phí cố định của cơ sở này ($-f_{old_fac}$).

Move 1 có ưu điểm là đơn giản, dễ triển khai và đặc biệt hiệu quả trong việc sửa các khách hàng bị gán vào kho có chi phí vận chuyển cao trong nghiệm Greedy. Tuy nhiên, nhược điểm là phạm vi tìm kiếm khá hẹp vì mỗi lần chỉ dịch chuyển đơn lẻ một khách hàng.

4.1.2 Move 2: Close facility (Đóng cơ sở)

Move 2 tập trung vào mục tiêu giảm thiểu chi phí cố định bằng cách thử đóng một cơ sở đang mở và phân bổ lại toàn bộ khách hàng của nó sang các cơ sở đang hoạt động khác. Để tăng xác suất gán thành công, các khách hàng trực thuộc cơ sở bị đóng sẽ được sắp xếp và xử lý theo thứ tự nhu cầu giảm dần. Phép move này giúp tiết kiệm được chi phí mở kho ($-f_i$), nhưng lại làm phát sinh chênh lệch chi phí phục vụ mới. Tổng Δ được tính bằng tổng của hai thành phần này. Ưu điểm của Move 2 là giảm đáng kể chi phí nếu nghiệm ban đầu mở quá nhiều cơ sở không cần thiết. Nhược điểm là tính khả thi phụ thuộc rất lớn vào lượng sức chứa dư thừa của các cơ sở đang mở khác, và phiên bản hiện tại chưa cho phép mở cơ sở mới để hứng tập khách hàng bị dời đi.

4.1.3 Move 3: Open + reassign (Mở và chuyển khách hàng)

Move 3 là dạng di chuyển kết hợp. Tại mỗi vòng lặp, thay vì chỉ xét một loại thay đổi, thuật toán tính toán và tìm kiếm nước đi có Δ âm tốt nhất từ hai nhóm: gán lại một khách hàng (tương tự Move 1) và đóng một cơ sở (tương tự Move 2). Việc kết hợp này mang lại sự linh hoạt cao hơn so với việc chạy lẻ tẻ từng move. Đổi lại, thuật toán phải chịu nhược điểm là chi phí tính toán cao hơn trong từng vòng lặp, đồng thời cấu trúc hiện tại chưa thực sự phản ánh được một chiến lược "mở cơ sở mới và tái phân bổ nhiều khách hàng" một cách toàn diện.

4.1.4 Move 4: Swap (Hoán đổi trạng thái đóng/mở)

Thay vì chuyển khách hàng đơn lẻ, Move 4 hoán đổi chéo vị trí phục vụ của hai khách hàng a và b thuộc hai cơ sở khác nhau. Phép hoán đổi này chỉ hợp lệ nếu dung lượng của cả hai cơ sở sau khi đổi vẫn không bị vi phạm. Do không làm thay đổi trạng thái đóng/mở của cơ sở, mức thay đổi chi phí Δ hoàn toàn là sự chênh lệch giữa tổng chi phí phục vụ mới và tổng chi phí phục vụ cũ. Move 4 cực kỳ hữu ích để vượt qua các tình huống một khách hàng bị kẹt do giới hạn sức chứa đơn lẻ. Dù vậy, nó cũng tồn tại nhược điểm là tốn nhiều thời gian xét do số lượng tổ hợp cặp khách hàng tăng theo bậc hai, và hoàn toàn không xử lý được vấn đề nghiệm ban đầu mở quá nhiều hoặc quá ít cơ sở.

4.2 Sơ đồ thuật toán Local Search

Thuật toán Local Search là quá trình lặp đi lặp lại việc tìm kiếm vùng lân cận để tối ưu hóa. Quy trình cụ thể diễn ra như sau:

1. Bắt đầu từ nghiệm khả thi khởi tạo bởi thuật toán Greedy 2, hệ thống tính toán hàm mục tiêu (tổng chi phí) của nghiệm hiện tại.
2. Khởi tạo vòng lặp cải thiện: Ở mỗi vòng lặp, duyệt qua không gian nghiệm lân cận được tạo ra bởi một trong các Move (1, 2, 3 hoặc 4).
3. Với từng nghiệm lân cận, kiểm tra điều kiện sức chứa và đánh giá chênh lệch chi phí Δ .
4. Nếu tồn tại các phép di chuyển hợp lệ có $\Delta < 0$, chọn ra move giúp giảm chi phí nhiều nhất (âm nhất) để áp dụng và cập nhật lại cấu hình nghiệm.

5. Lặp lại quá trình cho đến khi $\Delta \geq 0$, tức là không tìm được bất kỳ sự cải thiện nào nữa. Thuật toán dừng lại và kết luận đây là nghiệm tối ưu cục bộ.

Phương pháp này tuy đơn giản và chạy nhanh hơn MILP, nhưng chất lượng nghiệm phụ thuộc nhiều vào giới hạn của từng loại Move. Việc kết hợp nhiều Move lại với nhau thường đem lại hiệu quả vượt trội hơn.

4.3 Tối ưu hóa tính toán: Phương pháp tính nhanh Delta

Trong Local Search, hệ thống phải đánh giá hàng ngàn phép thử nghiệm ở mỗi vòng lặp. Nếu như sau mỗi nước đi, thuật toán tính lại toàn bộ tổng chi phí phục vụ và chi phí mở kho từ đầu, hiệu suất sẽ suy giảm trầm trọng. Để giải quyết vấn đề này, dự án áp dụng phương pháp "tính nhanh Delta".

Ý tưởng cốt lõi là chỉ tính toán phần chi phí cục bộ bị ảnh hưởng trực tiếp bởi phép Move. Tổng chi phí sau đó được cập nhật cực kỳ nhanh chóng thông qua phép cộng đơn giản: $Tngchiphmi = Tngchiphc + \Delta$.

- **Move 1:** Δ chỉ xét sự thay đổi cước vận chuyển của 1 khách hàng và chi phí đóng/mở của tối đa 2 cơ sở liên quan.
- **Move 2:** Δ chỉ cộng dồn chênh lệch chi phí phục vụ của tập khách hàng bị chuyển đi và trừ đi chi phí cố định của kho bị đóng.
- **Move 3:** Không sinh cấu trúc nghiệm mới, chỉ lưu lại giá trị Δ nhỏ nhất cùng thông tin move tương ứng từ tập hợp các tùy chọn của Move 1 và Move 2.
- **Move 4:** Δ thuần túy là độ biến thiên chi phí phục vụ chéo giữa hai khách hàng, tính toán chỉ trong vài phép tính cộng trừ.

Lợi ích của Delta nhanh là giúp giảm thiểu triệt để chi phí tính toán, làm cho việc so sánh các phương án trở nên trực quan và hỗ trợ cập nhật nghiệm tức thời. Tuy nhiên, hạn chế và thách thức lớn nhất khi áp dụng kỹ thuật này là đòi hỏi tính cẩn thận cực cao trong việc cài đặt. Hệ thống bắt buộc phải đồng bộ hóa chính xác sức chứa còn lại (*remaining_capacity*), trạng thái đóng/mở (*is_open*) và số lượng khách hàng của từng cơ sở ngay sau khi một move được phê duyệt để tránh việc thuật toán chấp nhận nhầm các move không khả thi ở những vòng lặp sau.

Chương 5

Cài đặt và thực nghiệm

Chương này trình bày chi tiết cách thức cài đặt chương trình và quy trình thực nghiệm cho bài toán Vị trí cơ sở có ràng buộc sức chứa (Capacitated Facility Location Problem - CFLP). Các thuật toán được triển khai trong dự án bao gồm mô hình Exact MILP, hai thuật toán Greedy và các biến thể Local Search để cải thiện nghiệm.

5.1 Mô tả bộ dữ liệu chuẩn

Dự án sử dụng bộ dữ liệu chuẩn **Capacitated warehouse location** từ **OR-Library**, một bộ dữ liệu phổ biến để đánh giá các thuật toán cho bài toán định vị cơ sở.

Dữ liệu gốc được lưu trong thư mục `data/` và sau khi tiền xử lý được lưu tại thư mục `data_preprocessed/`. Các tệp dữ liệu được sử dụng trong thực nghiệm bao gồm: `cap41.txt` đến `cap44.txt`, `cap51.txt`, và các tệp từ `cap61.txt` đến `cap134.txt`. Mỗi tệp biểu diễn một instance chứa các thông tin: số lượng cơ sở, số lượng khách hàng, sức chứa cơ sở, chi phí cố định, nhu cầu khách hàng và ma trận chi phí phục vụ.

Mô hình toán học được xây dựng dựa trên các ký hiệu sau:

- I : tập cơ sở, J : tập khách hàng.
- f_i : chi phí cố định khi mở cơ sở i .
- s_i : sức chứa của cơ sở i .
- d_j : nhu cầu của khách hàng j .
- c_{ij} : chi phí phục vụ khách hàng j từ cơ sở i .

Hàm mục tiêu và các ràng buộc tổng quát:

- **Hàm mục tiêu:** Tối thiểu hóa tổng chi phí $Z = \sum f_i y_i + \sum \sum c_{ij} x_{ij}$.
- **Ràng buộc phân bổ duy nhất:** $\sum x_{ij} = 1, \forall j \in J$.
- **Ràng buộc sức chứa:** $\sum d_j x_{ij} \leq s_i y_i, \forall i \in I$.
- **Ràng buộc mở cơ sở:** $x_{ij} \leq y_i$.

Quá trình đọc dữ liệu được đảm nhiệm bởi tệp `readfile.py` với hàm `parse_cap_file(filepath)` trả về một dictionary cấu trúc thống nhất làm đầu vào cho các thuật toán.

5.2 Cài đặt mô hình Exact MILP sử dụng thư viện OR-Tools

Mô hình Exact MILP được cài đặt tại tệp `run_experiments.py` sử dụng thư viện **OR-Tools** với bộ giải tuyến tính nguyên hỗn hợp **CBC** (`ortools.linear_solver`). Hàm chính `solve_for_baseline(data, time_limit=60)` giới hạn thời gian giải là 60 giây cho mỗi instance.

Mô hình sử dụng hai nhóm biến nhị phân:

- $y_i \in \{0, 1\}$: nhận giá trị 1 nếu mở cơ sở i , ngược lại bằng 0.
- $x_{ij} \in \{0, 1\}$: nhận giá trị 1 nếu khách hàng j gán cho cơ sở i , ngược lại bằng 0.

Mục đích chính của MILP là làm phương pháp baseline để so sánh với các thuật toán heuristic. Với giới hạn 60 giây, bộ giải sẽ trả về nghiệm tối ưu hoặc nghiệm khả thi tốt nhất tìm được và lưu tại `output/baselines_milp.csv`.

5.3 Cài đặt các thuật toán Greedy

Dự án triển khai hai chiến lược Greedy để khởi tạo nghiệm nhanh chóng:

- **Greedy 1** (`greedy1.py`): Ưu tiên mở các cơ sở có tỷ lệ lợi ích/chi phí tốt nhất. Phương pháp này nhanh và dễ cài đặt nhưng mang tính cục bộ cao, dễ chọn nhầm cơ sở ở những bước đầu, kết quả được lưu tại `output/results_greedy1.csv`.
- **Greedy 2** (`greedy2.py`): Tiếp cận theo hướng ngược lại, ưu tiên gán những khách hàng "khó" (nhu cầu lớn) trước vào cơ sở có chi phí gán tốt nhất. Phương pháp này giảm rủi ro vi phạm sức chứa và thường cho nghiệm ban đầu ổn định hơn, kết quả lưu tại `output/results_greedy2.csv`. Nghiệm của Greedy 2 được dùng làm điểm xuất phát cho các thuật toán Local Search.

5.4 Cài đặt thuật toán Local Search

Các biến thể Local Search được cài đặt để cải thiện nghiệm ban đầu do Greedy 2 tạo ra bằng cách đánh giá mức thay đổi chi phí Δ . Một move chỉ được chấp nhận nếu $\Delta < 0$ và không vi phạm sức chứa.

- **Move 1 - Reassignment** (`greedy2m1.py`): Thử gán lại một khách hàng từ cơ sở hiện tại sang một cơ sở mới để tối ưu chi phí phục vụ.
- **Move 2 - Close facility** (`greedy2m2.py`): Thử đóng một cơ sở đang hoạt động và chuyển toàn bộ khách của nó sang các cơ sở khác nhằm triệt tiêu chi phí cố định.
- **Move 3 - Open + reassign** (`greedy2m3.py`): Thử mở thêm một cơ sở đang đóng và chuyển khách hàng sang cơ sở mới này nếu lợi ích tiết kiệm đủ bù đắp chi phí mở.
- **Move 4 - Swap** (`greedy2m4.py`): Khám phá lân cận bằng cách hoán đổi chéo cơ sở phục vụ của 2 khách hàng, hữu ích khi các dịch chuyển đơn lẻ bế tắc do giới hạn sức chứa.
- **VND - Variable Neighborhood Descent** (`greedy2_vnd.py`): Kết hợp nhiều vùng lân cận (Move 1 đến Move 4), nếu một move hết khả năng cải thiện, thuật toán chuyển sang loại move tiếp theo, đem lại nghiệm chất lượng hơn các move chạy đơn lẻ.

5.5 Quy trình chạy thực nghiệm và Tổng hợp kết quả

Dự án quản lý tập trung toàn bộ quá trình chạy thông qua tệp `run_all_unified.py`. Quy trình chuẩn bao gồm các bước: đọc dữ liệu, chạy MILP baseline, chạy Greedy 1 và 2, khởi tạo các phép Move 1-4 cùng thuật toán VND trên nền Greedy 2, xuất dữ liệu độc lập và cuối cùng gộp vào tệp `output/results_all_unified.csv`.

Tệp báo cáo tổng hợp chứa các trường thông tin: *Objective* (hàm mục tiêu), *Time* (thời gian chạy), *Facilities Opened* (số cơ sở mở) và *Status* (trạng thái nghiệm) của từng phương pháp. Cách tổ chức tách bạch từng tệp (`preprocess_data.py`, `run_experiments.py`, v.v.) giúp quy trình chạy thực nghiệm trở nên khoa học, dễ bảo trì, và thuận tiện cho việc đánh giá khoảng cách nghiệm giữa phương pháp heuristic với nghiệm chính xác.

Chương 6

Đánh giá và so sánh kết quả

Chương này trình bày phần đánh giá kết quả thực nghiệm của các phương pháp đã cài đặt cho bài toán **Capacitated Facility Location Problem (CFLP)**. Các phương pháp được đưa lên bàn cân so sánh bao gồm: Exact MILP, Greedy 1, Greedy 2, Greedy 2 kết hợp với các Move (1, 2, 3, 4) và thuật toán VND.

Kết quả thực nghiệm được trích xuất từ tệp tổng hợp `output/results_all_unified.csv` dựa trên 37 instance đánh giá. Các tiêu chí đánh giá trọng tâm bao gồm:

- Chất lượng nghiệm, thể hiện qua giá trị hàm mục tiêu.
- Thời gian thực thi thuật toán.
- Mức cải thiện của Local Search so với nghiệm Greedy ban đầu.
- Hiệu quả của phương pháp tính nhanh delta.

6.1 So sánh chất lượng nghiệm giữa Exact MILP, Greedy và Local Search

Chất lượng nghiệm được đánh giá dựa trên giá trị hàm mục tiêu (bao gồm chi phí mở cơ sở và chi phí phục vụ khách hàng). Vì CFLP là bài toán tối thiểu hóa, objective càng nhỏ thì chất lượng nghiệm càng tốt. Phương pháp **Exact MILP** được sử dụng làm baseline định chuẩn.

Bảng dưới đây thống kê kết quả trung bình trên 37 instance:

6.1.1 So sánh MILP và Greedy

MILP có objective trung bình thấp nhất, đúng với vai trò là phương pháp chính xác. Đối với các thuật toán tham lam:

- **Greedy 1** có gap trung bình lên tới 25.01%, chứng tỏ chiến lược ưu tiên mở cơ sở theo tỷ lệ lợi ích/chi phí có tốc độ nhanh nhưng chất lượng nghiệm chưa ổn định do tính cục bộ.
- **Greedy 2** ưu việt hơn hẳn với gap chỉ 6.35%, nhờ việc ưu tiên gán các khách hàng có nhu cầu lớn trước, giúp hạn chế rủi ro thiếu hụt sức chứa ở các bước sau. Do đó,

Phương pháp	Objective trung bình	Gap trung bình so với MILP
MILP	984,851.64	0.00%
Greedy 1	1,222,660.52	25.01%
Greedy 2	1,050,819.52	6.35%
Move 1	1,041,582.58	5.50%
Move 2	1,049,876.81	6.25%
Move 3	1,040,604.24	5.42%
Move 4	1,034,945.37	4.88%
VND	1,030,497.12	4.45%

Bảng 6.1: Chất lượng nghiệm trung bình của các phương pháp

Greedy 2 là nền tảng khởi tạo lý tưởng cho Local Search.

6.1.2 So sánh Greedy 2 và các phương pháp Local Search

Các thuật toán Local Search đều cải thiện được nghiệm của Greedy 2 nhưng ở các mức độ khác nhau:

- **Move 1** (Gap 5.50%): Cải thiện tốt bằng cách gán lại những khách hàng chưa được tối ưu chi phí trong nghiệm Greedy ban đầu.
- **Move 2** (Gap 6.25%): Có tiềm năng tạo cải thiện lớn nếu đóng được cơ sở đắt đỏ, nhưng bị ràng buộc mạnh bởi sức chứa dư thừa của các cơ sở khác.
- **Move 3** (Gap 5.42%): Hiệu quả khi nghiệm ban đầu mở thiếu cơ sở, buộc khách hàng phải đi xa.
- **Move 4** (Gap 4.88%): Phép hoán đổi mang lại kết quả vượt trội hơn các move đơn lẻ, khai phá được các thay đổi phức tạp.
- **VND** (Gap 4.45%): Tốt nhất trong nhóm heuristic do kết hợp xoay vòng nhiều lần cận, giảm thiểu rủi ro mắc kẹt tại cực trị cục bộ.

6.1.3 Thống kê số instance đạt nghiệm tốt nhất và mức cải thiện

Khi chỉ xét riêng nhóm heuristic, thuật toán **VND** áp đảo khi đạt được cấu hình nghiệm tốt nhất trên 35/37 instance. Trong khi đó, Move 1 và Move 3 đạt best trên 15 instance, Move 2 và Move 4 đạt 13 instance, Greedy 2 đạt 13 instance và Greedy 1 chỉ vón vẹn 2 instance.

Về khả năng cải thiện trực tiếp so với nghiệm khởi tạo (Greedy 2): Move 4 và VND cùng cải thiện thành công trên 24 instance, Move 1 và Move 3 cải thiện được 19 instance, riêng Move 2 chỉ cải thiện được 7 instance.

6.2 So sánh thời gian thực thi của các phương pháp

Thời gian thực thi trung bình trên 37 instance được ghi nhận như sau:

MILP chạy chậm nhất (0.1921s) do phải xây dựng và giải mô hình nguyên hỗn hợp với số lượng biến nhị phân ($I + I \times J$) và các ràng buộc phức tạp. Ngược lại, **Greedy 2** có

Phương pháp	Thời gian trung bình (giây)
MILP	0.1921
Greedy 1	0.0038
Greedy 2	0.0001
Move 1	0.0005
Move 2	0.0001
Move 3	0.0008
Move 4	0.0010
VND	0.0019

Bảng 6.2: Thời gian thực thi trung bình của các phương pháp

tốc độ xử lý nhanh nhất (0.0001s) nhờ việc chỉ sắp xếp và phân bổ khách hàng theo tiêu chí định sẵn.

Các phương pháp Local Search có tốc độ chậm hơn Greedy 2 một chút nhưng vẫn chỉ ở mức một phần nghìn giây. VND dù phải duyệt qua nhiều cấu trúc lân cận nhưng vẫn duy trì thời gian trung bình cực kỳ ấn tượng (0.0019s), đem lại sự cân bằng hoàn hảo giữa thời gian chạy và chất lượng lời giải.

6.3 Thảo luận về hiệu quả các bước Local Search và phương pháp tính nhanh

Sự thành công về mặt tốc độ của thuật toán Local Search được đóng góp phần lớn nhờ vào **phương pháp tính nhanh delta**. Nếu tính lại từ đầu toàn bộ hàm mục tiêu ($\sum fixed_cost + \sum assignment_cost$), thuật toán sẽ phải liên tục quét lại toàn bộ dữ liệu hệ thống gây lãng phí tài nguyên nghiêm trọng.

Bằng cách sử dụng độ chênh lệch $\Delta = chiphsaumove - chiphtrcmove$, hệ thống chỉ thực hiện tính toán trên phần chi phí bị ảnh hưởng trực tiếp. Ví dụ: ở Move 1, phần Δ chỉ đơn thuần là $(c_{new_fac,j} - c_{old_fac,j})$ cộng trừ thêm chi phí đóng mở cơ sở nếu có. Nhờ đó, thuật toán Local Search và VND có thể đánh giá hàng ngàn nghiệm lân cận trong thời gian cực nhỏ (từ 0.0001s đến 0.0019s).

Về đặc thù từng toán tử:

- **Move 1 & Move 3** là các giải pháp tinh chỉnh phổ biến, cải thiện tốt các trường hợp Greedy gán khách hàng chưa tối ưu hoặc mở thiếu cơ sở.
- **Move 2** rất khó khả thi trên thực tế vì việc rút toàn bộ khách từ một kho thường vượt quá mức sức chứa dư thừa của các kho còn lại.
- **Move 4** mạnh mẽ trong việc vượt qua các cực trị cục bộ đơn giản nhờ khả năng thay đổi phối hợp đồng thời.
- **VND** hội tụ sức mạnh của tất cả các toán tử trên, giúp hạn chế rủi ro mắc kẹt.

6.4 Kết luận chương

Thông qua các số liệu thực nghiệm, dự án rút ra những kết luận trọng tâm sau:

1. MILP mang lại chất lượng nghiệm tốt nhất, đóng vai trò hoàn hảo làm baseline so sánh.
2. Greedy 2 vượt trội hơn Greedy 1 nhờ chiến lược xử lý khách hàng khó trước, tạo ra nghiệm ban đầu chất lượng và ổn định.
3. Thuật toán VND kết hợp Local Search là phương pháp heuristic xuất sắc nhất của dự án, đạt nghiệm tốt nhất trên 35/37 instance và kéo giảm khoảng cách so với MILP xuống chỉ còn 4.45%.
4. Áp dụng triệt để kỹ thuật tính nhanh Delta giúp các biến thể Local Search duy trì tốc độ tính toán chớp nhoáng.

Tóm lại, nếu yêu cầu độ chính xác tuyệt đối, MILP là lựa chọn duy nhất; nhưng nếu bài toán thực tế đòi hỏi sự cân bằng giữa tốc độ và chất lượng, mô hình Greedy 2 kết hợp VND là phương pháp hiệu quả và tối ưu nhất.

Chương 7

Kết luận và hướng phát triển

7.1 Tóm tắt toàn bộ đề án

Đề án đã nghiên cứu và cài đặt thành công các phương pháp giải cho bài toán vị trí cơ sở có ràng buộc sức chứa (Capacitated Facility Location Problem - CFLP), một bài toán tối ưu tổ hợp đặc biệt quan trọng trong lĩnh vực logistics và quản lý chuỗi cung ứng. Mục tiêu trọng tâm là lựa chọn tập cơ sở cần mở và phân bổ khách hàng sao cho tổng chi phí (gồm phí mở cơ sở và phí phục vụ) là nhỏ nhất, đồng thời thỏa mãn chặt chẽ các giới hạn về sức chứa.

Bài toán được tiếp cận theo hai hướng chính:

- Phương pháp chính xác (Exact Method) sử dụng mô hình **Mixed-Integer Linear Programming (MILP)** thông qua thư viện **OR-Tools** (với bộ giải **CBC**) để làm baseline định chuẩn.
- Các phương pháp xấp xỉ (Heuristics) bao gồm **Greedy**, **Local Search** và **Variable Neighborhood Descent (VND)**.

Về thuật toán khởi tạo, đề án cài đặt hai chiến lược tham lam: **Greedy 1** (ưu tiên mở cơ sở theo tỷ lệ lợi ích/chi phí) và **Greedy 2** (ưu tiên gán khách hàng "khó" có nhu cầu lớn trước). Kết quả cho thấy Greedy 2 tạo ra nghiệm ban đầu chất lượng hơn hẳn nhờ xử lý tốt ràng buộc sức chứa.

Trên nền tảng nghiệm của Greedy 2, đề án áp dụng Local Search với 4 bước di chuyển cục bộ:

- **Move 1 - Reassignment:** Gán lại khách hàng sang cơ sở khác.
- **Move 2 - Close facility:** Đóng cơ sở đang mở và tái phân bổ.
- **Move 3 - Open + reassign:** Mở thêm cơ sở nếu có lợi.
- **Move 4 - Swap:** Hoán đổi chéo cơ sở phục vụ giữa hai khách hàng.

Sự kết hợp xoay vòng của các toán tử trên tạo thành thuật toán **VND**, đồng thời việc tính toán được tối ưu tốc độ triệt để bằng kỹ thuật **tính nhanh Delta**.

Thực nghiệm trên bộ dữ liệu chuẩn **Capacitated warehouse location** (OR-Library) khẳng định **VND** là phương pháp heuristic xuất sắc nhất của dự án, đem lại chất lượng nghiệm tiệm cận MILP trong khi vẫn duy trì thời gian thực thi ở mức một phần nghìn

giây.

7.2 Đóng góp chính của đề án

1. **Xây dựng và cài đặt mô hình toán học MILP đầy đủ:** Đề án đã mô hình hóa bài toán với các biến quyết định, hàm mục tiêu và hệ thống ràng buộc chặt chẽ, sử dụng OR-Tools làm chuẩn mực đối sánh (baseline).
2. **Triển khai thuật toán Greedy hiệu quả:** Chứng minh được chiến lược ưu tiên khách hàng có nhu cầu cao (Greedy 2) phù hợp và hiệu quả hơn việc tiếp cận theo hướng ưu tiên lợi ích cơ sở (Greedy 1) trong môi trường có ràng buộc sức chứa.
3. **Thiết kế hệ thống Tìm kiếm cục bộ đa dạng:** Cài đặt thành công 4 loại Move khai thác các vùng lân cận khác nhau và tổng hợp chúng vào khung thuật toán VND.
4. **Tối ưu hóa hiệu suất bằng Delta nhanh:** Đưa ra kỹ thuật chỉ tính toán độ chênh lệch chi phí, giúp giảm thiểu triệt để thời gian chạy của Local Search so với việc tính lại từ đầu.
5. **Thực nghiệm minh bạch, khoa học:** Xây dựng hệ thống code kiểm thử đồng bộ trên bộ dữ liệu OR-Library, cho phép so sánh khách quan và trực diện sức mạnh của từng phương pháp.

7.3 Hạn chế của đề án

Mặc dù đạt được những kết quả khả quan, đề án vẫn còn một số điểm giới hạn:

- **Hạn chế của mô hình MILP:** Khó mở rộng với các instance lớn do số lượng biến nhị phân và ràng buộc tăng theo hàm mũ, dẫn đến thời gian giải quá lâu.
- **Tính cục bộ của các thuật toán heuristic:** Các toán tử Local Search hiện tại (kể cả VND) vẫn khá đơn giản, dễ bị phụ thuộc vào chất lượng của nghiệm khởi tạo và thứ tự áp dụng vùng lân cận, dẫn đến rủi ro mắc kẹt ở cực trị cục bộ.
- **Vắng bóng Metaheuristic nâng cao:** Đề án mới chỉ dừng lại ở nhóm thuật toán Local Search cơ bản mà chưa khai thác các siêu heuristic (Metaheuristic) mạnh mẽ hơn như Simulated Annealing, Genetic Algorithm hay Tabu Search.
- **Thực nghiệm thiếu góc nhìn đa chiều:** Báo cáo hiện tại mới chỉ tập trung vào Objective và Time, chưa phân tích sâu vào tỷ lệ lấp đầy sức chứa, độ ổn định nghiệm hay trực quan hóa dữ liệu qua biểu đồ.

7.4 Hướng phát triển trong tương lai

Dựa trên các hạn chế đã phân tích, đề án đề xuất các hướng phát triển mở rộng như sau:

7.4.1 Nâng cấp khung Local Search hiện tại

- **Cải tiến tiêu chí Greedy:** Bổ sung thêm các tham số như số lượng cơ sở có thể phục vụ khách hàng, hoặc độ chênh lệch chi phí phục vụ (penalty) để phân loại mức độ "khó" của khách hàng một cách toàn diện hơn.
- **Thiết kế các Move phức tạp hơn:** Chuyển đồng thời một nhóm khách hàng, kết hợp đóng/mở nhiều kho cùng lúc để mở rộng vùng lân cận.
- **Nâng cấp lên GVNS (General Variable Neighborhood Search):** Bổ sung thêm bước "làm nhiễu" (shaking) để ép thuật toán văng ra khỏi các vùng tối ưu cục bộ, tạo tiền đề để tìm kiếm sâu hơn.

7.4.2 Áp dụng các Metaheuristic nâng cao

- **Simulated Annealing (SA):** Sử dụng cơ chế nhiệt độ để chấp nhận các nghiệm tạm thời xấu hơn với xác suất $P = \exp(-\Delta/T)$, giúp hệ thống vượt qua các thung lũng cục bộ.
- **Genetic Algorithm (GA):** Khởi tạo một quần thể nghiệm và thực hiện lai ghép (crossover), đột biến (mutation) theo danh sách cơ sở phục vụ của từng khách. Cần tích hợp thêm hàm sửa lỗi (repair) để xử lý các cá thể vi phạm ràng buộc sức chứa.
- **Tabu Search:** Sử dụng danh sách cấm (Tabu list) để lưu lại các nước đi vừa thực hiện (như cấm mở lại kho vừa đóng), giúp hệ thống bị ép phải khám phá các vùng không gian mới thay vì quay vòng vô ích.

7.4.3 Tiếp cận lai (Matheuristics) và mở rộng phân tích

Kết hợp sức mạnh của Heuristic và Exact Method (MILP): dùng Heuristic để cung cấp nghiệm khởi tạo cực tốt cho MILP, hoặc dùng MILP để giải tối ưu một nhóm bài toán con đã được Heuristic khoanh vùng. Đồng thời, các nghiên cứu tiếp theo cần bổ sung hệ thống biểu đồ trực quan, mở rộng tập dữ liệu kiểm thử quy mô lớn và phân tích chi tiết hành vi của thuật toán.